

Sentrix Litepaper

An open-source, EVM-compatible Layer-1 written in Rust.

This is the short read. If you build software, you can finish it in a coffee break and know whether Sentrix is worth your time. The deep version — formal consensus proofs, performance model, failure handling — lives in the whitepaper. This document is meant to be read by a human, not audited by one.

The 30-second version

You already know how to build on Sentrix.

It runs the EVM. Your Solidity compiles unchanged. MetaMask connects. Hardhat deploys. Foundry tests. `ethers` and `viem` talk to it — in fact `viem` already ships Sentrix as a built-in chain, so you can target it without writing a single chain definition:

```
import { sentrix, sentrixTestnet } from "viem/chains";
```

Under that familiar surface is a chain built from scratch in Rust: one binary, one local database, BFT finality in a single block, and a monetary policy that no one — not the founder, not a DAO — can change after genesis.

That's the pitch. The rest of this document is why each of those choices was made, and what you actually get.

Why another L1?

Most “new chain” pitches start with a number — more TPS, lower fees, some benchmark. Sentrix doesn't lead with a number, because a number you can't verify is marketing.

It leads with three properties a builder can check in an afternoon:

1. **It's the EVM you already know — not a lookalike.** Sentrix executes through `revm`, the Rust EVM. There is no transpiler, no “Solidity-compatible” asterisk, no custom toolchain to learn. If your contract runs on Ethereum, it runs here.
2. **It's open source and written in one language.** The entire node — consensus, execution, state, networking — is one Rust workspace at github.com/sentrix-labs/sentrix. You can read it, audit it, and run it as a single process. No JVM, no Go-plus-C-plus-shell-scripts, no five-service docker stack to stand up a node.
3. **The economics are fixed in code, not in a roadmap.** Maximum supply, block reward, halving cadence, and fee burn are protocol constants. There is no governance lever to inflate the supply later. What you read in the tokenomics section is what the chain enforces — forever.

Sentrix is not trying to be the fastest chain or the cheapest. It is trying to be the one a developer can fully understand, run themselves, and trust not to move the goalposts.

What you can build today

Anything you'd build on an EVM chain. DEXes, tokens, NFT marketplaces, on-chain games, DeFi primitives — the contracts are identical. Point your existing deploy script at a Sentrix RPC URL and go.

On two rails, your choice. Sentrix has a native token rail (`TokenOp`) and the standard EVM rail (ERC-20). The native rail is a fixed-ABI, gas-free-ish flat-fee path for simple fungible tokens — cheap and deterministic.

The EVM rail is the full programmable surface. Most teams use the EVM rail; the native rail exists for high-volume, low-complexity token operations that don't need a contract VM.

With the tooling already in place:

You need	Sentrix has
Wallet	MetaMask works out of the box (add network, or use the <code>viem</code> chain def)
Deploy / test	Hardhat, Foundry, ethers , viem — unchanged
Block explorer	scan.sentrixchain.com (EIP-3091)
Contract verification	verify.sentrixchain.com (Sourcify)
Testnet funds	faucet.sentrixchain.com
Canonical contracts	@sentrix-labs/canonical-contracts on npm
Real-time data	JSON-RPC, gRPC, and WebSocket (<code>eth_subscribe</code>) endpoints

How it works, without the proofs

Sentrix's consensus is called **Voyager**: delegated proof-of-stake to pick who proposes, and a three-phase Byzantine-fault-tolerant vote (propose → prevote → precommit) to finalize. The important consequence for you as a builder:

A block is final the moment it commits. There are no probabilistic confirmations, no “wait for 12 blocks,” no reorg risk once a transaction lands. When your transaction is in a committed block, it is permanent. Target block time is one second.

Execution is **dual-rail and deterministic**: every honest node running the same blocks against the same state produces a bit-identical result. State is committed through a binary sparse Merkle tree, so the chain can prove any account's balance or any contract's storage at any height.

The node is **one Rust binary over one embedded key-value store (MDBX)**. An operator runs a single process. There is no separate execution client and consensus client to keep in sync — the thing that has caused a generation of infrastructure pain on other chains simply doesn't exist here.

That's the architecture in three paragraphs. The whitepaper turns each of these into formal safety and liveness conditions if you want to verify rather than trust.

SRX in 60 seconds

SRX is the native coin. It pays fees, secures the chain through staking, and is the unit validators earn.

Property	Value
Symbol	SRX (smallest unit: sentri , 1 SRX = 100,000,000 sentri)
Native decimals	8 (Bitcoin-style accounting; wrapped WSRX is standard 18-decimal ERC-20)
Maximum supply	315,000,000 SRX — fixed, not governable
Premine	63,000,000 SRX (20% of cap), public on-chain across four named accounts
Block reward	1 SRX, halving every 126,000,000 blocks (~4 years, Bitcoin-cadence)

Property	Value
Native fee	flat 0.0001 SRX — 50% burned, 50% to the validator
EVM fee	standard EIP-1559 gas

Two design choices worth calling out:

- **Half of every native fee is burned.** It leaves the supply permanently. As usage grows, the burn becomes a real counterweight to issuance.
- **Block rewards are earned by the validators who *signed* the previous block, pro-rata by stake — not just the proposer.** This ties revenue to doing the work of consensus, not to winning the proposer lottery.

The full distribution, vesting, and allocation policy lives in the tokenomics document. The protocol-level constants above are the part that's set in stone.

Run it yourself

A validator is one binary and one key. You stake SRX, you sign blocks, you earn rewards; if you go offline or double-sign, you get slashed. The active set is selected by stake.

Point a node at the network, sync, and you're a fullnode serving RPC. Stake above the minimum and register, and you're a validator candidate. There is deliberately no permission gate beyond stake — the whole node is open source, so anyone can run the exact software the chain runs.

```
git clone https://github.com/sentrix-labs/sentrix
cargo build --release
./target/release/sentrix start --genesis genesis/testnet.toml
```

Where things stand

Developers trust honesty more than hype, so here is the real state:

- **Live on mainnet:** Voyager DPoS+BFT consensus, the EVM rail, the native token rail, staking and slashing, reward distribution, the explorer, the Sourcify verifier, and the public RPC / gRPC / WebSocket surface.
- **Live on testnet, baking toward mainnet:** ongoing consensus hardening and the state-in-trie migration (SIP-6). Testnet is where new consensus code proves itself before it touches mainnet.
- **Shipped in code, not yet activated:** native NFT operations (SRC-721/1155) and several fork-gated features that wait behind environment flags until a testnet bake says they're safe.

Sentrix ships behind fork gates on purpose: consensus changes are dark-launched in the binary, baked on testnet, and only then activated on mainnet by a coordinated restart. Nothing consensus-critical flips on mainnet without that path.

Read next

- **Deep dive:** Sentrix Whitepaper — formal consensus protocol, performance model, adversarial bounds, failure handling.
- **Code:** github.com/sentrix-labs/sentrix
- **Docs:** docs.sentrixchain.com

- **Explorer:** scan.sentrrixchain.com
- **Testnet faucet:** faucet.sentrrixchain.com

Chain IDs: **7119** (mainnet), **7120** (testnet).